

Компьютерная графика

Практика 4: Индексированный рендеринг,
перспективная проекция, буфер глубины

2026

- Простой текстовый формат описания 3D моделей
- Обычно используется для старых моделей, для простоты чтения и хранения
- Довольно неоптимален по занимаемой памяти (потому что текстовый)
- Поддерживает нормали, текстурные координаты, и простейшие материалы
- В заготовке практики уже есть загрузчик OBJ файлов

- В этой практике нельзя менять код шейдеров!
- Все преобразования модели (сдвиги, повороты, и т.п.) и настройки камеры нужно выполнять через матрицы `model`, `view` и `projection`
- В C++ матрицы заполняем по строкам, в Rust – по столбцам (так устроены используемые библиотеки матриц)
- **NB:** математическими библиотеками (`glam` в Rust, `math` в C++) пользоваться пока нельзя :)

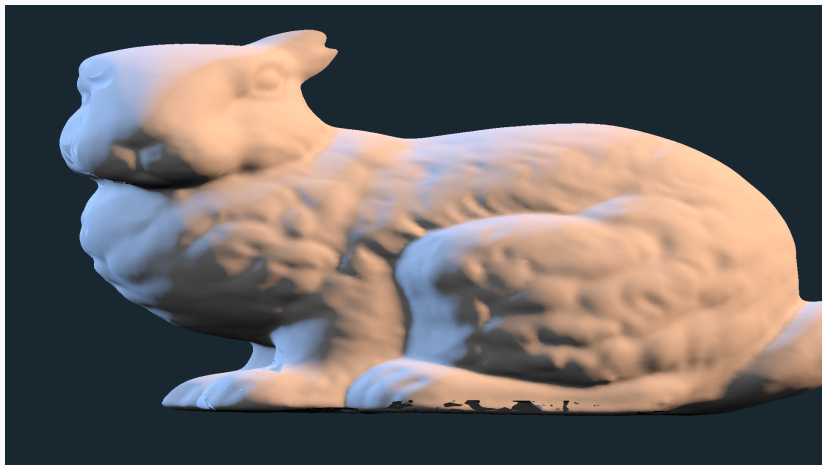


Задание 1

Рисуем зайца

- В объекте `bunny` уже загружены вершины и индексы 3D модели
- Настраиваем атрибуты вершин в `RenderPipelineDescriptor`'е (см. поля структуры `ObjVertex` – нам нужны только первые два)
- Создаём буферы для вершин (`usage = Vertex`) и индексов (`usage = Index`), загружаем в них данные (один раз, на старте программы)
- При рисовании устанавливаем `render_pass`'у наши буферы:
`render_pass.set_vertex_buffer()`,
`render_pass.set_index_buffer()`
- Рисуем при помощи `render_pass.draw_indexed()`

Задание 1



Задание 2

Вращаем зайца

- Меняем матрицу `model` чтобы заяц крутился в **плоскости XZ**
- В качестве угла нужно взять что-нибудь зависящее от времени, например `angle = time;`
- Заяц будет обрезаться по $z \in [0, 1]$ (особенно хорошо видно, что обрезаются уши и часть головы)
- Отмасштабируем его *по всем осям* `scale = 0.5f`, тоже с помощью матрицы `model`

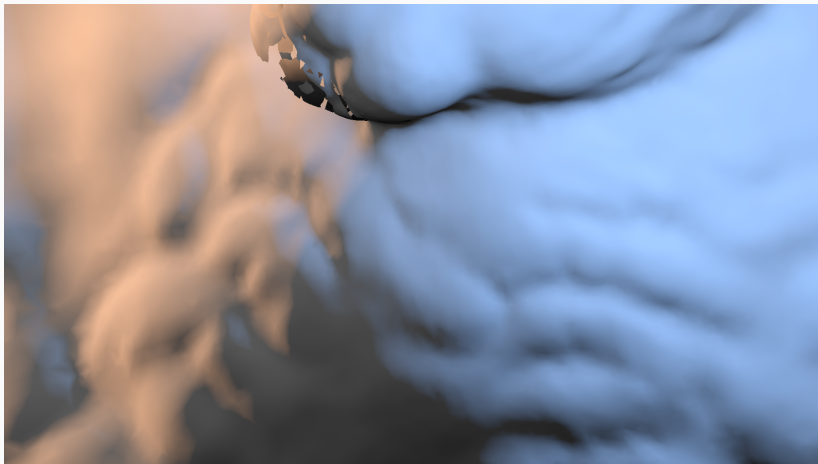
Задание 2



Добавляем перспективу

- Выбираем значения **near, far, top, right**
 - **near** – маленький, но не слишком, в духе $0.001 \dots 0.1$
 - **far** – большой, но не слишком, в духе $10.0 \dots 1000.0$
 - Отношение **right/near** – тангенс половины угла обзора, например **right = near** соответствует углу обзора в 90°
 - Отношение **right/top** – aspect ratio экрана (**width/height**)
- В матрицу **projection** записываем матрицу проекции с использованием выбранных значений (см. слайд с лекции)
- Заяц будет виден изнутри

Задание 3



Задание 4

Включаем тест глубины

- Сдвигаем камеру по оси Z на какое-то расстояние (например, на 2-3 единицы), меняя матрицу **view**
 - N.B.: Камера смотрит в сторону -Z, поэтому *сдвиг камеры* должен быть в сторону *положительной* оси Z
 - N.B.: При этом матрица **view** содержит *обратное преобразование!*
- Заяц будет рисоваться неправильно: задние грани перекрывают передние
- Настраиваем тест глубины в `RenderPipelineDescriptor`:
`format = Depth24Plus, depthWriteEnabled = true` (в C++ `WGPUOptionalBool_True`), `depthCompare = Less`

Задание 4

Создаём буфер глубины

- Заводим объекты `depthBuffer`: `WGPUTexture` и `depthBufferView`: `WGPUTextureView`
- Их нужно создать на старте + пересоздавать каждый раз, когда изменились размеры экрана
- `depthBuffer` создаём через `device.create_texture`, параметры:
`usage = RenderAttachment, dimension = 2D,`
`size = {width, height, 1}, format = Depth24Plus`
- `depthBufferView` создаём через `depthBuffer.create_view`, параметры: `usage = RenderAttachment, dimension = 2D,`
`format = Depth24Plus, aspect = DepthOnly, mipLevelCount = 1,`
`arrayLayerCount = 1`
- Добавляем `depthStencilAttachment` в наш `renderPass`:
`view = depthBufferView, depthLoadOp = Clear,`
`depthStoreOp = Discard, depthClearValue = 1.0,`
`depthReadOnly = false`

Задание 4

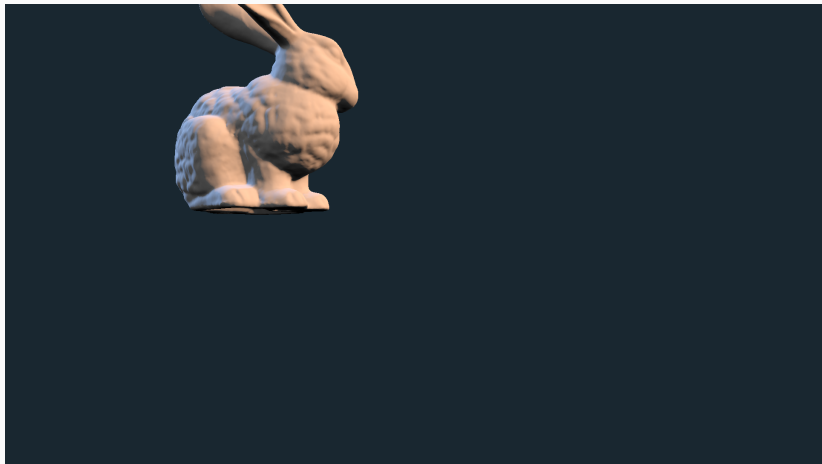


Задание 5

Двигаем зайца

- Заведём переменные `bunny_x` и `bunny_y` с координатами центра зайца по X и Y
- Изменим матрицу `model`, добавив соответствующие сдвиги по X и Y
- Перед рисованием каждого кадра обновим положение зайца:
 - Если нажата клавиша влево (`SDLK_LEFT` в C++ / `KeyCode::ArrowLeft` в Rust), сдвинем зайца влево:
`bunny_x -= speed * dt`
 - Аналогично `RIGHT`, `DOWN`, `UP`
 - Состояние нажатости клавиш доступно в словаре `keydown` / `app.keydown`

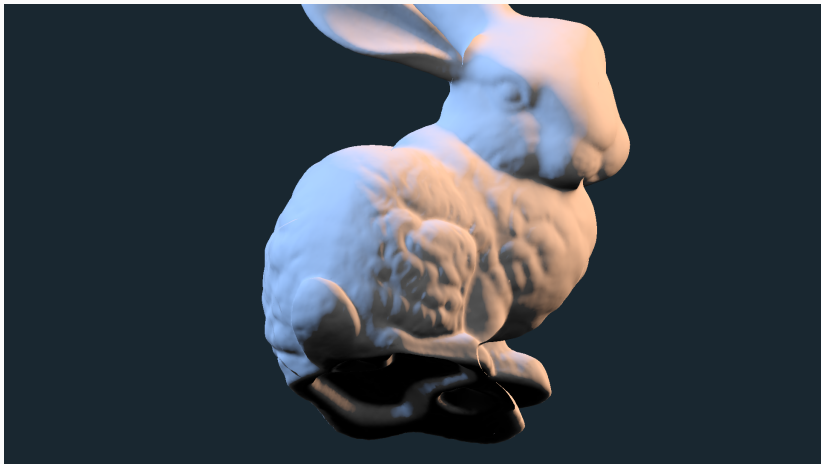
Задание 5



Играем с face culling

- Включим back-face culling: установим `renderPipelineDescriptor.primitive.cullMode = Back`
- Ничего не изменится: заяц сделан так, чтобы все треугольники были CCW
- Изменим режим на **Front**
- Должны быть видны задние грани зайца и не видны передние
- Выглядеть будет странно – не пугайтесь, попробуйте увидеть и понять, что происходит :)

Задание 6



Задание 7*

Три вращающихся зайца

- Рисуем три зайца в разных местах, вращающихся в плоскостях XY, XZ, YZ соответственно
- Три раза настраиваем матрицу `model` + делаем `render_pass.set_immediates()` + рисуем `render_pass.draw_indexed()`
- Всё ещё можно двигать стрелочками, т.е. должны учитываться `bunny_x` и `bunny_y` (для одного зайца, или для всех – как хотите)

Задание 7*

